

# REST API

## Complete Technical Reference Guide

---

API Concepts • API Tokens • Communication • Architecture • Implementation

***Use Case: E-Commerce Order Management System***

# 1. What is an API?

API stands for Application Programming Interface. It is a set of rules and protocols that allows one software application to communicate with another. Think of an API as a waiter in a restaurant — you (the client) tell the waiter (the API) what you want, the waiter takes your request to the kitchen (the server), and brings back your food (the response).

### Key Definition

API = A contract between two software systems that defines:

- What requests can be made (endpoints)
- How to make those requests (methods, headers, body)
- What responses will be returned (format, status codes)
- What rules govern access (authentication, rate limits)

## 1.1 Types of APIs

| API Type      | Description & Use Case                                                    |
|---------------|---------------------------------------------------------------------------|
| REST API      | Uses HTTP, stateless, resource-based URLs. Most popular for web services. |
| GraphQL API   | Query language for APIs; client specifies exact data shape needed.        |
| SOAP API      | XML-based protocol; used in enterprise/banking systems.                   |
| WebSocket API | Full-duplex communication; ideal for real-time apps like chat.            |
| gRPC API      | Google's RPC framework; uses Protocol Buffers; fast & efficient.          |

# 2. REST API — Deep Dive

REST (Representational State Transfer) is an architectural style for designing networked applications. A REST API (also called RESTful API) uses HTTP requests to perform CRUD (Create, Read, Update, Delete) operations on resources.

## 2.1 REST Principles (6 Constraints)

| # | Principle         | Meaning                          | Example                                          |
|---|-------------------|----------------------------------|--------------------------------------------------|
| 1 | Stateless         | Each request is self-contained   | No server-side sessions; auth in every request   |
| 2 | Client-Server     | Separation of concerns           | Frontend (React) talks to Backend (Node.js)      |
| 3 | Cacheable         | Responses can be cached          | Cache GET /products for 5 minutes                |
| 4 | Uniform Interface | Consistent URL & method patterns | GET /orders, POST /orders, DELETE /orders/1      |
| 5 | Layered System    | Client unaware of backend layers | Load balancer, API gateway, service — all hidden |
| 6 | Code on Demand    | Server can send executable code  | Sending JS scripts (optional)                    |

## 2.2 HTTP Methods in REST

REST uses standard HTTP verbs to define the action being performed on a resource:

| Method | URL Pattern  | Action             | Request Body    | Response             |
|--------|--------------|--------------------|-----------------|----------------------|
| GET    | /orders      | Fetch all orders   | None            | 200 + list of orders |
| GET    | /orders/{id} | Fetch one order    | None            | 200 + single order   |
| POST   | /orders      | Create new order   | Order JSON      | 201 + new order      |
| PUT    | /orders/{id} | Replace full order | Full order JSON | 200 + updated order  |
| PATCH  | /orders/{id} | Update partial     | Partial JSON    | 200 + updated order  |
| DELETE | /orders/{id} | Delete order       | None            | 204 No Content       |

## 2.3 HTTP Status Codes

| Status Code      | Meaning & When Used                     |
|------------------|-----------------------------------------|
| 200 OK           | Request succeeded (GET, PUT, PATCH)     |
| 201 Created      | Resource created successfully (POST)    |
| 204 No Content   | Success but no body returned (DELETE)   |
| 400 Bad Request  | Invalid input / missing required fields |
| 401 Unauthorized | Missing or invalid authentication token |
| 403 Forbidden    | Authenticated but not allowed to access |

|                   |                                   |
|-------------------|-----------------------------------|
| 404 Not Found     | Resource does not exist           |
| 422 Unprocessable | Validation failed on request body |
| 429 Too Many Req  | Rate limit exceeded               |
| 500 Server Error  | Unexpected server-side failure    |

## 3. API Token — Authentication & Security

An API Token (also called API Key or Bearer Token) is a unique identifier used to authenticate and authorize requests to an API. It proves that the requester is who they claim to be and that they have permission to access the requested resource.

### 3.1 How API Token Authentication Works

#### Step-by-Step Token Flow

|        |                                                                          |
|--------|--------------------------------------------------------------------------|
| Step 1 | Client registers / logs in with credentials (username + password)        |
| Step 2 | Server verifies credentials and generates a signed token (JWT or opaque) |
| Step 3 | Server sends the token back to the client in the response body           |
| Step 4 | Client stores the token (localStorage, cookie, memory)                   |
| Step 5 | Client sends token in every subsequent API request header                |
| Step 6 | Server validates the token on each request before processing             |
| Step 7 | If valid → process request. If invalid/expired → return 401              |

### 3.2 Types of API Tokens

| Token Type         | Description                                                                      |
|--------------------|----------------------------------------------------------------------------------|
| API Key            | Simple static string. Sent via header or query param. Easy but less secure.      |
| Bearer Token (JWT) | JSON Web Token. Self-contained, signed, has expiry. Industry standard.           |
| OAuth 2.0 Token    | Authorization framework. Issues access + refresh tokens. Used by Google, GitHub. |
| Basic Auth         | Base64 encoded username:password. Simple but insecure without HTTPS.             |

**Session Token**

Server-side session ID. Stateful. Used in traditional web apps.

### 3.3 JWT Token Structure

A JWT (JSON Web Token) has three parts separated by dots: Header.Payload.Signature

**JWT Token Breakdown**

```
// JWT Example
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9      ← Header (algorithm & type)
.
eyJ1c2VySWQiOiIxMjMiLCJyb2xlIjoieWRTaW4iLCJleHAiOjE3MDAwMDB9 ← Payload (claims)
.
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c ← Signature (HMAC-SHA256)

// Decoded Payload:
{
  "userId": "123",
  "role": "admin",
  "email": "user@example.com",
  "iat": 1700000000, // issued at
  "exp": 1700003600 // expires at (+1 hour)
}
```

### 3.4 Sending the API Token in Requests

The most common and secure way to send an API token is in the HTTP Authorization header:

**Token in HTTP Request Headers**

```
// Method 1: Bearer Token (most common)
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

// Method 2: API Key in header
X-API-Key: your-api-key-here

// Method 3: API Key in query param (AVOID — appears in logs)
GET /orders?api_key=your-key

// Method 4: Basic Auth (Base64 encoded)
Authorization: Basic dXNlcjpwYXNzd29yZA==
```

**Security Best Practices for API Tokens**

- ✓ Always use HTTPS — never send tokens over plain HTTP
- ✓ Set short expiry times (15 min to 1 hour for access tokens)
- ✓ Use refresh tokens to silently renew access tokens
- ✓ Store tokens securely — HttpOnly cookies or secure memory (not localStorage for sensitive apps)
- ✓ Implement token rotation on every refresh
- ✓ Revoke tokens server-side on logout
- ✓ Never log tokens in server logs
- ✓ Use rate limiting per token to prevent abuse

## 4. API Communication — How It Works

API communication is the process of a client sending an HTTP request to a server, the server processing it, and returning an HTTP response. Every API call follows this standard request-response cycle.

### 4.1 The Request-Response Cycle

**CLIENT → REQUEST → SERVER → PROCESS → RESPONSE → CLIENT**

#### Anatomy of an HTTP Request

```
HTTP Request — POST /api/v1/orders
POST /api/v1/orders HTTP/1.1
Host: api.shopzone.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json
Accept: application/json
X-Request-ID: req-abc-123

{
  "customerId": "CUST-001",
  "items": [
    { "productId": "PROD-789", "quantity": 2, "price": 29.99 },
    { "productId": "PROD-456", "quantity": 1, "price": 49.99 }
  ],
  "shippingAddress": {
    "street": "123 Main St",
    "city": "Hyderabad",
    "pincode": "500001"
  },
  "paymentMethod": "CARD"
}
```

#### Anatomy of an HTTP Response

```
HTTP Response — 201 Created
HTTP/1.1 201 Created
Content-Type: application/json
X-Request-ID: req-abc-123
```



```
X-RateLimit-Remaining: 99

{
  "success": true,
  "data": {
    "orderId": "ORD-20241118-001",
    "status": "CONFIRMED",
    "total": 109.97,
    "createdAt": "2024-11-18T10:30:00Z",
    "estimatedDelivery": "2024-11-22"
  },
  "message": "Order placed successfully"
}
```

## 4.2 API Communication Flow — E-Commerce Use Case

Below is the complete communication flow for placing an order in our e-commerce system:

| # | Actor         | Action                    | Detail                                      |
|---|---------------|---------------------------|---------------------------------------------|
| 1 | Browser       | User clicks 'Place Order' | React app collects cart + address data      |
| 2 | React App     | POST /api/v1/orders       | Sends JWT token + order payload as JSON     |
| 3 | API Gateway   | Validates JWT token       | Checks signature, expiry, rate limit        |
| 4 | Order Service | Validates order payload   | Checks product availability, pricing        |
| 5 | Order Service | Saves order to database   | INSERT into orders table, generate order ID |
| 6 | Order Service | Publishes event to queue  | Sends 'ORDER_CREATED' event to RabbitMQ     |
| 7 | Email Service | Consumes queue event      | Sends confirmation email to customer        |
| 8 | Order Service | Returns 201 response      | Sends back order ID, status, total          |
| 9 | React App     | Shows success screen      | Displays order confirmation to user         |

## 4.3 API Communication Patterns

| Pattern      | Description & Use Case                                                      |
|--------------|-----------------------------------------------------------------------------|
| Synchronous  | Client waits for response. Standard REST calls. Used for CRUD operations.   |
| Asynchronous | Client sends request and moves on. Server processes in background (queues). |

|                    |                                                                             |
|--------------------|-----------------------------------------------------------------------------|
| Polling            | Client repeatedly calls API to check status: GET /orders/{id}/status        |
| Webhooks           | Server calls client URL when event happens. Used for payment confirmations. |
| Long Polling       | Server holds connection open until data is ready. Near real-time updates.   |
| Server-Sent Events | Server pushes events to client over single HTTP connection (SSE).           |

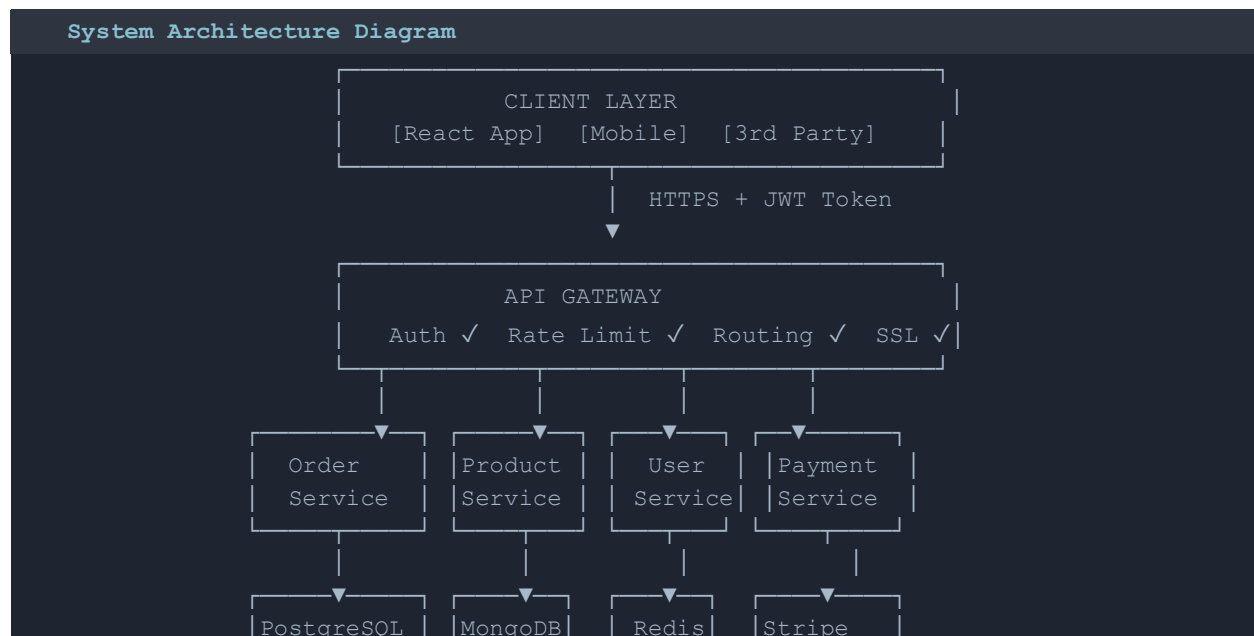
## 5. System Architecture

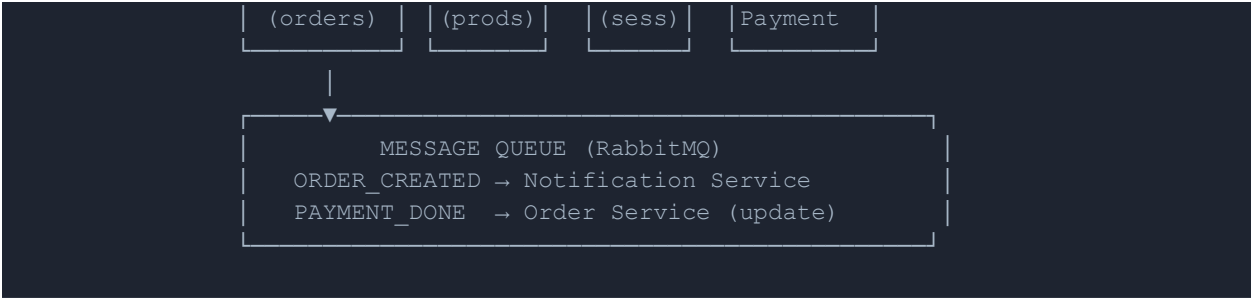
The architecture of our E-Commerce REST API follows a layered microservices pattern with an API Gateway as the single entry point. This ensures separation of concerns, scalability, and security.

### 5.1 Architecture Layers

|                               |                                                                                                                      |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Layer 1: Client</b>        | React Web App / Mobile App / Third-party API Consumer. Sends HTTP requests with JWT tokens.                          |
| <b>Layer 2: API Gateway</b>   | Single entry point. Handles: Authentication, Rate Limiting, Load Balancing, Request Routing, SSL Termination.        |
| <b>Layer 3: Microservices</b> | Order Service, Product Service, User Service, Payment Service, Notification Service — each independently deployable. |
| <b>Layer 4: Data Layer</b>    | PostgreSQL (orders), MongoDB (products), Redis (cache/sessions), Elasticsearch (search).                             |
| <b>Layer 5: Message Queue</b> | RabbitMQ / Kafka for async inter-service communication. Decouples services.                                          |

### 5.2 Architecture Diagram (Text Representation)





### 5.3 API Gateway Responsibilities

| Responsibility  | How It Works                                                                           |
|-----------------|----------------------------------------------------------------------------------------|
| Authentication  | Validates JWT on every request. Rejects 401 if missing/expired.                        |
| Authorization   | Checks user role/scope. Returns 403 if insufficient permissions.                       |
| Rate Limiting   | Limits requests per token (e.g., 100 req/min). Returns 429 if exceeded.                |
| Load Balancing  | Distributes traffic across multiple service instances (round-robin).                   |
| Request Routing | Routes <code>/orders</code> → Order Service, <code>/products</code> → Product Service. |
| SSL Termination | Decrypts HTTPS at gateway level. Internal services use HTTP.                           |
| Logging         | Logs all requests with correlation ID for debugging and auditing.                      |
| Caching         | Caches frequent GET responses (e.g., product catalog) at gateway level.                |

## 6. Implementation — Step by Step

We will implement a complete REST API for an E-Commerce Order Management System using Node.js, Express, JWT authentication, and PostgreSQL.

### 6.1 Project Setup

#### Project Initialization

```
# Step 1: Initialize project
mkdir ecommerce-api && cd ecommerce-api
npm init -y

# Step 2: Install dependencies
npm install express jsonwebtoken bcryptjs dotenv cors helmet
npm install pg sequelize express-rate-limit express-validator
npm install morgan uuid

# Step 3: Install dev dependencies
npm install --save-dev nodemon jest supertest

# Step 4: Project structure
ecommerce-api/
├── src/
│   ├── config/           (db.js, auth.js)
│   ├── controllers/      (orderController.js, authController.js)
│   ├── middleware/       (auth.js, rateLimit.js, validate.js)
│   ├── models/           (Order.js, User.js, Product.js)
│   ├── routes/           (orders.js, auth.js, products.js)
│   └── app.js
├── .env
└── server.js
```

### 6.2 Environment Configuration (.env)

#### .env — Environment Variables

```
PORT=3000
DB_HOST=localhost
DB_NAME=ecommerce
DB_USER=postgres
DB_PASSWORD=secret
JWT_SECRET=your-super-secret-jwt-key-min-32-chars
JWT_EXPIRES_IN=1h
JWT_REFRESH_EXPIRES_IN=7d
```

```
RATE_LIMIT_WINDOW_MS=60000
RATE_LIMIT_MAX=100
```

## 6.3 Authentication Middleware (JWT Validation)

### src/middleware/auth.js — JWT Middleware

```
// src/middleware/auth.js
const jwt = require('jsonwebtoken');

const authenticate = (req, res, next) => {
  try {
    // 1. Extract token from header
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({
        success: false,
        error: 'No token provided. Use: Authorization: Bearer <token>'
      });
    }

    // 2. Verify token signature and expiry
    const token = authHeader.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // 3. Attach user info to request
    req.user = decoded; // { userId, role, email }
    next();

  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({ success: false, error: 'Token expired' });
    }
    return res.status(401).json({ success: false, error: 'Invalid token' });
  }
};

// Role-based authorization
const authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ success: false, error: 'Access denied' });
  }
  next();
};

module.exports = { authenticate, authorize };
```

## 6.4 Login Endpoint — Generate Token

### src/controllers/authController.js — Login & Token Generation

```
// src/controllers/authController.js
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');

const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    // 1. Find user in database
    const user = await User.findOne({ where: { email } });
    if (!user) return res.status(401).json({ error: 'Invalid credentials' });

    // 2. Verify password hash
    const isMatch = await bcrypt.compare(password, user.passwordHash);
    if (!isMatch) return res.status(401).json({ error: 'Invalid credentials' });

    // 3. Generate Access Token (short-lived)
    const accessToken = jwt.sign(
      { userId: user.id, email: user.email, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: process.env.JWT_EXPIRES_IN } // 1 hour
    );

    // 4. Generate Refresh Token (long-lived)
    const refreshToken = jwt.sign(
      { userId: user.id },
      process.env.JWT_SECRET + '_refresh',
      { expiresIn: process.env.JWT_REFRESH_EXPIRES_IN } // 7 days
    );

    // 5. Return tokens
    res.status(200).json({
      success: true,
      accessToken,
      refreshToken,
      expiresIn: 3600,
      user: { id: user.id, email: user.email, role: user.role }
    });

  } catch (err) {
    res.status(500).json({ error: 'Server error' });
  }
};
```

## 6.5 Order API Endpoints

### src/controllers/orderController.js — All CRUD Endpoints

```
// src/controllers/orderController.js
const { v4: uuidv4 } = require('uuid');

// GET /api/v1/orders — List all orders for authenticated user
const getOrders = async (req, res) => {
  const orders = await Order.findAll({ where: { customerId: req.user.userId } });
  res.json({ success: true, count: orders.length, data: orders });
};

// GET /api/v1/orders/:id — Get single order
const getOrder = async (req, res) => {
  const order = await Order.findOne({
    where: { id: req.params.id, customerId: req.user.userId }
  });
  if (!order) return res.status(404).json({ error: 'Order not found' });
  res.json({ success: true, data: order });
};

// POST /api/v1/orders — Create new order
const createOrder = async (req, res) => {
  const { items, shippingAddress, paymentMethod } = req.body;

  const total = items.reduce((sum, i) => sum + i.price * i.quantity, 0);

  const order = await Order.create({
    id:          uuidv4(),
    customerId:   req.user.userId,
    items:        JSON.stringify(items),
    shippingAddress: JSON.stringify(shippingAddress),
    paymentMethod,
    total,
    status:       'PENDING',
  });

  res.status(201).json({ success: true, data: order });
};

// PATCH /api/v1/orders/:id — Update order status
const updateOrderStatus = async (req, res) => {
  const order = await Order.findByPk(req.params.id);
  if (!order) return res.status(404).json({ error: 'Not found' });
  await order.update({ status: req.body.status });
  res.json({ success: true, data: order });
};

// DELETE /api/v1/orders/:id — Cancel order
```



```
const deleteOrder = async (req, res) => {
  const order = await Order.findByPk(req.params.id);
  if (!order) return res.status(404).json({ error: 'Not found' });
  await order.destroy();
  res.status(204).send();
};

module.exports = { getOrders, getOrder, createOrder, updateOrderStatus,
deleteOrder };
```

## 6.6 Route Definitions

```
src/routes/orders.js + src/app.js

// src/routes/orders.js
const express = require('express');
const router = express.Router();
const { authenticate, authorize } = require('../middleware/auth');
const ctrl = require('../controllers/orderController');

// All order routes require authentication
router.use(authenticate);

router.get('/', ctrl.getOrders); // GET /orders
router.get('/:id', ctrl.getOrder); // GET /orders/:id
router.post('/', ctrl.createOrder); // POST /orders
router.patch('/:id/status', authorize('admin'), ctrl.updateOrderStatus); // PATCH /orders/:id/status (admin only)
router.delete('/:id', ctrl.deleteOrder); // DELETE /orders/:id

module.exports = router;

// src/app.js
const express = require('express');
const helmet = require('helmet');
const cors = require('cors');
const rateLimit = require('express-rate-limit');
const app = express();

app.use(helmet()); // Security headers
app.use(cors()); // CORS
app.use(express.json()); // JSON body parsing
app.use(rateLimit({ windowMs: 60000, max: 100 })); // Rate limiting

app.use('/api/v1/auth', require('../routes/auth'));
app.use('/api/v1/orders', require('../routes/orders'));
app.use('/api/v1/products', require('../routes/products'));
```

```
module.exports = app;
```

## 6.7 Testing the API with curl

### Testing REST API with curl Commands

```
# Step 1: Login and get token
curl -X POST http://localhost:3000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"user@example.com","password":"secret"}'

# Response: { accessToken: 'eyJ...', refreshToken: '...' }

# Step 2: Create an order (use token from Step 1)
curl -X POST http://localhost:3000/api/v1/orders \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." \
  -H "Content-Type: application/json" \
  -d '{
    "items": [{"productId":"P1","quantity":2,"price":29.99}],
    "shippingAddress": {"street":"123 Main","city":"Hyderabad"},
    "paymentMethod": "CARD"
  }'

# Step 3: Get all orders
curl http://localhost:3000/api/v1/orders \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

# Step 4: Get specific order
curl http://localhost:3000/api/v1/orders/ORD-001 \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

# Step 5: Delete order
curl -X DELETE http://localhost:3000/api/v1/orders/ORD-001 \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

## 7. Error Handling & API Response Standards

### 7.1 Standard Response Format

#### Standard API Response Format

```
// ✓ Success Response Format
{
  "success": true,
  "data":    { ... },           // the actual payload
  "message": "Order created",  // optional human message
  "meta":    { "page": 1, "total": 150 } // pagination (if needed)
}

// ✗ Error Response Format
{
  "success": false,
  "error":   "Validation failed",
  "details": [
    { "field": "items", "message": "Items array cannot be empty" },
    { "field": "paymentMethod", "message": "Must be CARD or UPI" }
  ],
  "requestId": "req-abc-123",
  "timestamp": "2024-11-18T10:30:00Z"
}
```

### 7.2 Global Error Handler

#### Global Error Handler Middleware

```
// src/middleware/errorHandler.js
const errorHandler = (err, req, res, next) => {
  console.error(`[${req.method}] ${req.path}`, err.message);

  // Sequelize validation errors
  if (err.name === 'SequelizeValidationError') {
    return res.status(422).json({
      success: false,
      error: 'Validation failed',
      details: err.errors.map(e => ({ field: e.path, message: e.message }))
    });
  }

  // JWT errors (handled in middleware, but fallback)
  if (err.name === 'JsonWebTokenError') {

```

```
    return res.status(401).json({ success: false, error: 'Invalid token' });
  }

  // Default 500
  res.status(err.status || 500).json({
    success: false,
    error: err.message || 'Internal server error',
    requestId: req.headers['x-request-id']
  });
};

module.exports = errorHandler;
```

## 8. Quick Reference Summary

### REST API Cheat Sheet

| Topic          | Key Points                                                      |
|----------------|-----------------------------------------------------------------|
| REST API       | Uses HTTP, stateless, resource-based, JSON format               |
| HTTP Methods   | GET=read, POST=create, PUT=replace, PATCH=update, DELETE=remove |
| Status Codes   | 2xx=success, 4xx=client error, 5xx=server error                 |
| API Token      | Sent in Authorization: Bearer <token> header                    |
| JWT Structure  | Header.Payload.Signature — base64 encoded, signed with secret   |
| Token Security | HTTPS always, short expiry, refresh tokens, HttpOnly cookies    |
| Architecture   | Client → API Gateway → Microservices → Database                 |
| Authentication | Verify JWT → Extract user → Attach to req.user → Next           |
| Authorization  | Check role in token → Allow or 403 Forbidden                    |
| Error Format   | { success, error, details, requestId, timestamp }               |
| Rate Limiting  | 100 req/min per token. 429 Too Many Requests if exceeded        |
| Versioning     | Always version your API: /api/v1/orders, /api/v2/orders         |